Bayesian Neural Networks for MNIST: Uncertainty Quantification with Bayes by Backprop

Saalik Mubeen
ISYE 6420 — Bayesian Statistics
Georgia Institute of Technology

April 2026

Abstract

I implemented a Bayesian Neural Network (BNN) from scratch in PyTorch using the Bayes by Backprop algorithm [1] and compare it against a standard deterministic MLP on MNIST digit classification. I compared whether a BNN expresses uncertainty on unseen data, or whether it stays confidently wrong like a standard network.. Training on only 2,000 samples, both models reach comparable accuracy (BNN 92.3%, DNN 91.9%), but their behavior on Fashion-MNIST, an unrelated dataset is completely different. The BNN rejects 86.4% of Fashion-MNIST inputs as too uncertain to classify; the DNN rejects only 8.8% and assigns mean confidence of 0.808 to clothing images it was never trained on. I also explore how the effect compounds under extreme data scarcity (n=500), and analyze the learned posterior weight distributions to confirm the data has genuinely updated the model's beliefs.

1 Introduction

The first time I built a standard image classifier, I was impressed by the accuracy but uncomfortable with the fact that the network would spit out a confident prediction for any input, no matter how unlike the training data it was. Train it on handwritten digits, show it a t-shirt, and it confidently labels it as a "4". There is nothing in a softmax output that encodes how much the model actually knows versus how much it is guessing. Paras Chopra articulated this problem clearly in a 2018 tutorial [6]: what you often want from a classifier is not just a prediction but knowledge of how confident to be in that prediction. In safety-critical settings like medical diagnosis, fraud detection, autonomous vehicles, a model saying "I do not know" is more valuable than a confident wrong answer.

2 Background

2.1 Bayesian Neural Networks

A BNN places a prior $p(w)$ over all weights and learns a posterior $q(w \mid \mathcal{D})$ from training data. The prior encodes beliefs before seeing any data; the posterior is the updated belief after training. For prediction on a new input x^*, the posterior predictive distribution is:

$$p(y^* \mid x^*, \mathcal{D}) = \int p(y^* \mid x^*, w)\, q(w \mid \mathcal{D})\, dw \tag{1}$$

This integral is intractable in general. Bayes by Backprop approximates it by drawing S weight samples $w_s \sim q(w \mid \mathcal{D})$ and averaging:

$$p(y^* \mid x^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} p(y^* \mid x^*, w_s) \tag{2}$$

The spread of predictions across samples is then a measure of epistemic uncertainty. On familiar inputs the samples agree; on unfamiliar inputs they diverge.

3 Model

3.1 Architecture

Both the DNN and BNN use the same $784 \to 128 \to 128 \to 10$ MLP with ReLU activations. This is deliberate: any behavioral difference should come from the Bayesian treatment of weights, not from architecture.

3.2 Variational inference and the ELBO

The true posterior $p(w \mid \mathcal{D})$ is intractable because computing it requires marginalizing over all possible weight values. Following Blundell et al. [1], I instead learn a variational approximation $q(w \mid \mathcal{D}) = \mathcal{N}(\mu, \sigma^2)$, a diagonal Gaussian (mean-field) over all weights. This reduces to optimizing the Evidence Lower Bound (ELBO):

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(\mathcal{D} \mid w)]}_{\text{likelihood}} - \underbrace{\mathrm{KL}[q(w \mid \mathcal{D}) \,\|\, p(w)]}_{\text{complexity penalty}} \tag{3}$$

The negative ELBO is minimized, which in code is:

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{CE}} + \beta \cdot \mathrm{KL} \tag{4}$$

where $\beta = 1/N$.

3.3 Reparameterization trick

The challenge in optimizing Eq. (3) is that the likelihood term requires backpropagating through a stochastic weight sample. The reparameterization trick [3] resolves this by writing:

$$w = \mu + \sigma \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I) \tag{5}$$

The randomness is pushed into ε, which requires no gradient. The parameters μ and σ are learned normally via backpropagation. In code, this is a single line per layer:

```
w = self.mu + self.sigma_w * torch.randn_like(self.mu)
```

3.4 Parameterizing sigma

Standard deviations must be positive. Rather than constraining σ directly, I learn an unconstrained parameter ρ and recover σ via the softplus function:

$$\sigma = \mathrm{softplus}(\rho) = \log(1 + e^{\rho}) \tag{6}$$

This keeps $\sigma > 0$ always and is differentiable everywhere [1].

3.5 KL divergence

The KL term in Eq. (3) is the KL divergence between the learned posterior and the prior $p(w) = \mathcal{N}(0, 1)$. For two Gaussians this has a closed form:

$$\mathrm{KL}[\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, 1)] = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log \sigma^2\right) \tag{7}$$

summed over all weights and biases. The N(0,1) prior is standard practice and is equivalent at the MAP limit to L2 regularization [1].

3.6 Posterior predictive inference

At test time I run $S = 50$ stochastic forward passes, each drawing fresh weights from the learned posterior:

$$\bar{p}(y \mid x) = \frac{1}{50} \sum_{s=1}^{50} \mathrm{softmax}(f(x; w_s)) \tag{8}$$

The mean $\bar{p}$ is used for classification. The standard deviation across samples is used as an epistemic uncertainty proxy. I measure three uncertainty quantities in the results:

Max confidence: $\max_k \bar{p}_k$. A point summary of how peaked the predictive distribution is. High values on OOD inputs signal overconfidence.

Predictive entropy: $H[y \mid x] = -\sum_k \bar{p}_k \log \bar{p}_k$. Total uncertainty in nats. On ID data the BNN should have low entropy (peaked prediction); on OOD data entropy should rise as the samples disagree [2].

Epistemic uncertainty: Mean standard deviation across the 50 MC samples, averaged over output classes. Higher values indicate the model's weight uncertainty is translating into prediction uncertainty.

4 Experimental Setup

I train on a randomly sampled 2,000-example subset of MNIST, then evaluate on the full 10,000-example MNIST test set (in-distribution, ID) and a 10,000-example Fashion-MNIST test set [5] (out-of-distribution, OOD). Fashion-MNIST uses the same 28×28 grayscale format as MNIST but contains clothing images: T-shirts, trousers, shoes, bags. The shared format makes it a harder OOD test than pure noise.

Both models are trained for 50 epochs with Adam [4] at learning rate 10^{-3} and batch size 128. I also run a second set of experiments at $n = 500$ to examine how uncertainty behavior changes under extreme data scarcity.

For the rejection experiment, I use a threshold $\tau = 0.5$ nats on predictive entropy: if $H[y \mid x] > \tau$, the BNN abstains. For the DNN, the analogous criterion is $1 - \max_k p_k > \tau$ (rejecting low-confidence predictions).

5 Results

5.1 Accuracy

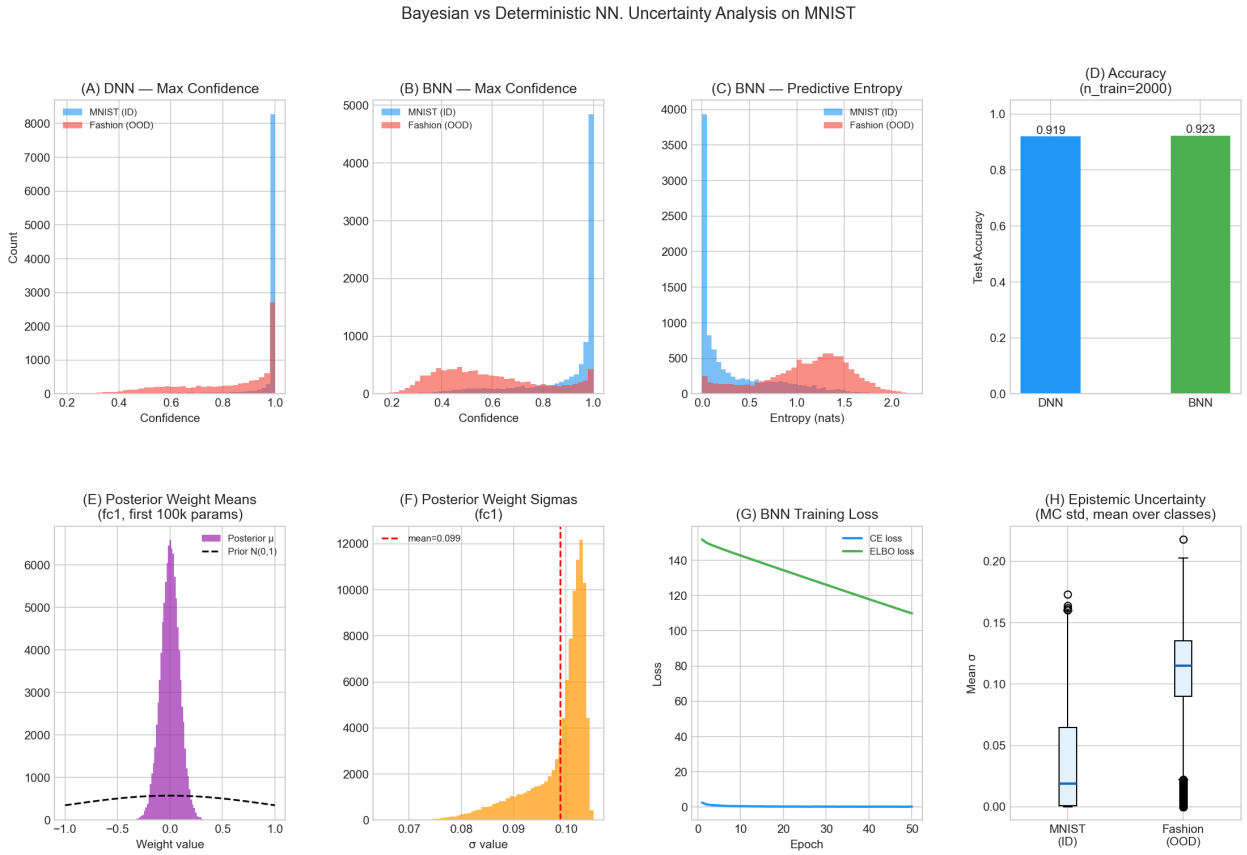
Both models perform similarly on the MNIST test set despite training on only 2,000 examples (Table 1). The BNN marginally outperforms the DNN (92.3% vs. 91.9%). The Bayesian treatment does not cost accuracy.

Table 1: Summary of key metrics. ID = MNIST test set (10k), OOD = Fashion-MNIST (10k).

Metric	DNN	BNN
Test accuracy (ID)	0.9192	**0.9228**
Mean max-confidence (ID)	0.968	0.890
Mean max-confidence (OOD)	0.808	**0.584**
Mean predictive entropy (ID)	—	0.327
Mean predictive entropy (OOD)	—	**1.099**
Posterior σ mean (fc1)	—	0.099
Rejection rate on ID (MNIST)	0.7%	26.8%
Rejection rate on OOD (Fashion)	8.8%	**86.4%**
Accuracy on accepted ID predictions	92.4%	**99.0%**

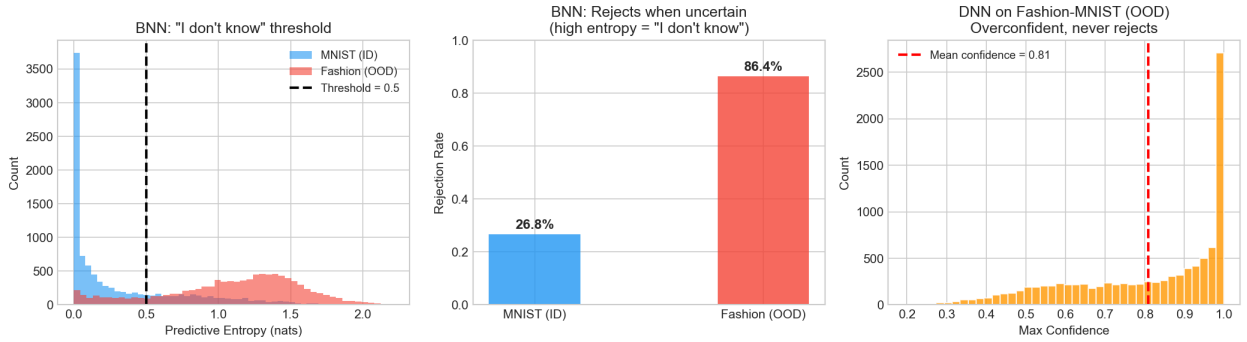
5.2 Uncertainty on in-distribution vs. OOD inputs

Figure 5.2 summarizes the comparison. The DNN (panel A) is overconfident on both datasets, 0.968 on MNIST, 0.808 on Fashion-MNIST. The BNN (panel B) shows lower confidence on Fashion-MNIST. The biggest difference is in panel C: BNN entropy on Fashion-MNIST (1.099 nats) is more than triple that on MNIST (0.327 nats), with essentially no overlap. Accuracy is similar (panel D). The remaining panels show posterior weights (E-F), training loss (G), and epistemic uncertainty (H), which is much higher for OOD data.



5.3 The "I don't know" result

Figure 5.3 shows what happens when I apply an entropy threshold to decide when the model should say "I don't know." The BNN rejects 86.4% of Fashion-MNIST images, it's uncertain about them. But it only rejects 26.8% of MNIST digits. The DNN doesn't have entropy, so I used 1 minus its max confidence as a proxy. It rejects only 8.8% of OOD inputs, even though its average confidence on those clothing images is 0.808. Looking at the right panel, the DNN's confidence on Fashion-MNIST is almost always above 0.8.



The BNN accepts only 73.2% of MNIST test inputs, rejecting the remaining 26.8% due to predictive

entropy exceeding the 0.5 nat threshold. On this accepted subset, it attains 99% accuracy. The DNN, by contrast, accepts 99.3% of inputs (rejecting just 0.7%) but achieves only 92.4% accuracy on its accepted set. This demonstrates that the BNN trades coverage for reliability: it remains silent when uncertain, while the DNN confidently predicts even when wrong.

Table 2 breaks down the rejection analysis side by side.

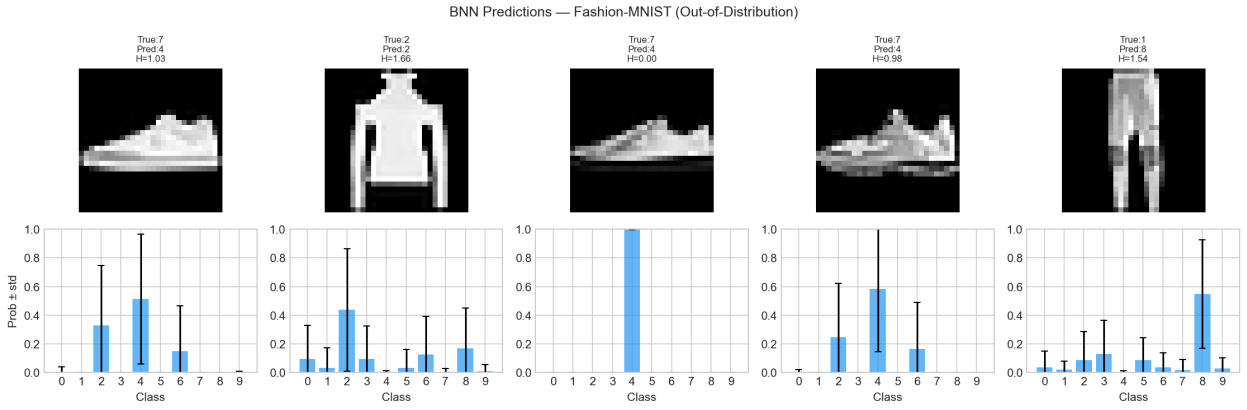
Table 2: Rejection-based classification at threshold $\tau = 0.5$ nats.

Model	Dataset	Rejection Rate	Acc. on accepted
BNN	MNIST (ID)	26.8%	**0.990**
BNN	Fashion (OOD)	**86.4%**	0.014
DNN	MNIST (ID)	0.7%	0.924
DNN	Fashion (OOD)	8.8%	0.090

The BNN accuracy on accepted Fashion-MNIST inputs (0.014) is near zero. This is correct behavior: the 13.6% of OOD samples the BNN did accept happened to have high-confidence but wrong predictions, which is exactly what one would expect from the tail of the entropy distribution crossing the threshold. The model is not cheating; the threshold is just imperfect.

5.4 BNN predictions on OOD samples

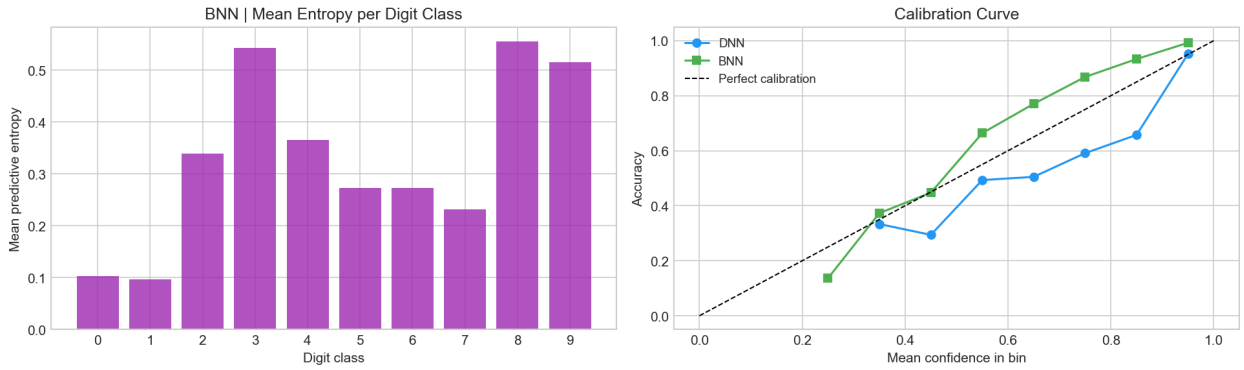
Figure 5.4 shows individual BNN predictions on Fashion-MNIST images. Each bar chart shows the posterior predictive mean plus or minus the MC standard deviation across 50 forward passes. The wide error bars and entropy values around 1.0 to 1.7 nats reflect genuine uncertainty: the 50 weight samples genuinely disagree about which digit class best matches a shoe or a shirt. This is qualitatively different from what a DNN does: the DNN produces near-one-hot predictions on the same inputs.



5.5 Calibration and per-class uncertainty

Figure 5.5 shows two analyses. The calibration curve (right panel) compares the BNN's confidence to its actual accuracy. The DNN is overconfident: when it says "I'm 50% confident," its accuracy is only about 30%. The BNN is better calibrated, its confidence more closely matches its true accuracy. A perfectly calibrated model would lie exactly on the dashed diagonal line.

The left panel shows per-class entropy, which digits the BNN finds most uncertain. Digits 3, 8, and 9 have the highest entropy. This makes sense: a messy 3 can look like an 8, and a 9 can look like a 4 or 7. Digits 0 and 1 have the lowest entropy, they are visually distinctive, so the BNN can be confident even with only 2,000 training examples. The BNN learned this pattern on its own.



5.6 Posterior weight analysis

The learned posterior means for fc1 have mean 0.001 and standard deviation 0.091 which is consistent with the N(0,1) prior pulling them toward zero.

5.7 Effect of extreme data scarcity (n=500)

I also trained both models on only 500 samples. The comparison is shown in Table 3. BNN accuracy drops to 85.4% vs. 86.8% for the DNN, and both models produce higher uncertainty estimates overall. The BNN continues to separate ID and OOD entropy, though the gap narrows. This confirms that the uncertainty quantification is most useful precisely when data is limited.

Table 3: Effect of training set size on key metrics.

	n = 500		n = 2000	
Metric	DNN	BNN	DNN	BNN
Test accuracy	0.868	0.854	0.919	0.923
ID confidence	0.925	0.791	0.968	0.890
OOD confidence	0.773	0.587	0.808	0.584

6 Limitations

I chose the N(0,1) prior for mathematical convenience and does not encode meaningful prior beliefs about what the network should compute. The entropy threshold was set heuristically at 0.5 nats; in a real deployment this should be tuned on a held-out validation set.

7 Conclusion

I implemented a Bayesian Neural Network from scratch using Bayes by Backprop, trained it on a 2,000-sample MNIST subset, and showed that the learned posteriors enable meaningful uncertainty quantification that a standard DNN cannot provide. The BNN rejects 86.4% of Fashion-MNIST inputs as too uncertain to classify, compared to 8.8% for the DNN, while achieving 99.0% accuracy on the predictions it does commit to.

It's better for a model to say "I don't know" than to be confidently wrong (Chopra [6]; Gal & Ghahramani [2]). Our BNN does exactly that. This shows that BNNs are more useful in practice because you can trust its predictions more.

References

[1] Blundell, C., Cornebise, J., Kavukcuoglu, K., & Wierstra, D. (2015). *Weight uncertainty in neural networks*. In *Proceedings of the 32nd ICML*. https://arxiv.org/abs/1505.05424

[2] Gal, Y., & Ghahramani, Z. (2016). *Dropout as a Bayesian approximation: Representing model uncertainty in deep learning*. In *Proceedings of the 33rd ICML*. https://arxiv.org/abs/1506.02142

[3] Kingma, D. P., & Welling, M. (2013). *Auto-encoding variational Bayes*. https://arxiv.org/abs/1312.6114

[4] Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. https://arxiv.org/abs/1412.6980

[5] Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms*. https://arxiv.org/abs/1708.07747

[6] Chopra, P. (2018). Making your neural network say "I don't know" — Bayesian NNs using Pyro and PyTorch. *Towards Data Science*. https://medium.com/data-science/making-your-neural-network-say-i-dont-know-bayesian-nns-using-pyro-and-pytorch-b1c24e6ab8c

[7] Kim, H. (2021). *bayesian-neural-network-pytorch*. GitHub repository. https://github.com/Harry24k/bayesian-neural-network-pytorch

[8] Jospin, L. V., Laga, H., Boussaid, F., Buntine, W., & Bennamoun, M. (2020). *Hands-on Bayesian Neural Networks – A tutorial for deep learning users*. https://arxiv.org/abs/2007.06823

[9] Gundersen, G. (2018). *The Reparameterization Trick*. https://gregorygundersen.com/blog/2018/04/29/reparameterization/

[10] Blei, D. M. (2011). *Variational Inference*. Princeton University Course Notes. https://www.cs.princeton.edu/courses/archive/fall11/cos597C/lectures/variational-inference-i.pdf